fil_023

Open Grant Proposal: Optimized multi-hop payments for the Filecoin Retrieval Market

Name of Project: Optimized multi-hop payments for the Filecoin Retrieval Market

Proposal Category: `technical-designn` – payments & collateral

Proposer: `andrew.gord.stewart`

Do you agree to open source all work you do on behalf of this RFP and dual-license under MIT and APACHE2 licenses?: Yes

Project Description

The Filecoin retrieval market will be a scalable, trustless content-delivery network where clients pay providers for content. The detailed roadmap for the retrieval market proposes an HTLC-backed 'mega-proxy' architecture for retrieval market micropayments. However, the HTLC approach requires the mega-proxy (aka intermediary) to process every micropayment, making it highly resource-intensive to operate at scale. Nitro channels are an advancement in state channels that require the intermediary's involvement only at the beginning and end of a stream of micropayments, with the parties otherwise exchanging tickets directly.

Nitro virtual channels could be used today to enable the Filecoin retrieval market. However, we propose a series of optimizations to the Nitro virtual channels that will decrease latency, cpu usage, and gas costs of disputes without sacrificing security or relying on trust. We believe that optimizing these metrics is critical for creating a large scale retrieval market. The three key metrics we optimize are:

1. The latency required for Bob to be able to redeem a payment from Alice through some intermediary Irene, *without relying on trust assumptions.* For the remainder of this document, we refer to this latency as "time to payment" (TTP).
2. The CPU burden placed on the actors, with special attention to the intermediary, who is expected to connect many `(client, provider)` pairs.
3. The gas cost to recover funds in a virtually funded channel via on-chain disputes, with special attention to the case where a `client` or `provider` going offline.

Background on Nitro virtual channels

Nitro virtual channels are described in detail in the research paper as well as this blog post. Nitro protocol is implemented on-chain for the Ethereum EVM and off-chain in Typescript. Nitro channels have been used in a PoC app called Web3Torrent, an incentivized torrent web-app. With Web3Torrent, a single on-chain deposit with an intermediary is enough to enable high-throughput conditional micropayments with any peer connected to the same intermediary. We have also used TLA+ modelling to show that participants can always gain control of their funds in a bounded amount of time regardless of the actions of other parties. We've invested time in pushing down gas costs by 4x. We've also worked with Consensys Diligence on a security audit and are now highly attuned to smart contract best-practices. By building on these successes, we will drive the three key performance

metrics down further – by more than 50% – while preserving or improving on the security and operational cost of our existing implementation.

Proposed work

In the last couple of months, we've developed a few ideas for how to optimize the virtual funding protocol (VFP). We believe the overhead of the protocol is minimal or close to minimal across three key metrics outlined above. In addition, our new construction enables a greatly simplified multi-hop payment protocol, with constant happy-path & sad-path complexity.

In this proposal, we ask for funding to:

- Carefully specify and document the new protocol to confirm the findings.
- Implement the optimized Nitro protocol in Solidity, and implement the off-chain VFP in Typescript.
- Formally verify optimized Nitro.

Value

The optimized Nitro virtual protocol will offer the following benefits, which we believe are important for a large scale retrieval market:

- The number of blocking network messages required to set up a virtual payment channel is reduced to a theoretical minimum, without introducing trust assumptions. Our optimal construction achieves our conjectured minimum of 2 sequential messages: one payer-to-intermediary message followed by two concurrent intermediary-to-payer and intermediary-to-receiver messages.
- We minimize the load placed on the intermediary.
- We minimize the on-chain costs for recovering funds during a dispute to ensure disputes are economically feasible.
- We continue to support generalized state channels which can be programmed to support conditional payments or other arbitrary logic, enabling the community to innovate novel cryptoeconomic incentives for retrieval markets. The intermediary is application-agnostic and can support any application channel with no modification.

Solving these problems will enable scalable micropayments for the Filecoin Retrieval Market.

What are the risks that will make executing on this project difficult?

- In Milestone 4, we plan to use Dafny (a verification-friendly programming language) to perform some formal verification of the target protocol, but our team does not have experience with Dafny. We have colleagues in Consensys Software Incorporated who have already done some formal verification work in Dafny for the Filecoin retrieval market. These colleagues are interested in bootstrapping us towards formally verifying parts of our protocol in Dafny, mitigating this risk.
- It is difficult to estimate deadlines for the deliverables for this project, due to the following factors:
  - this is protocol-heavy work with some uncertainties remaining
  - the later milestones of this project partially depend on the outcomes of the earlier milestones
  - we anticipate an increase in travel & vacation due to ongoing changes to travel policy

Deliverables

Our target for this project is:

1. Milestone 1 — a specification of the optimized Nitro protocol. This specification will include all relevant details required to implement Nitro protocol eg. in Filecoin.

2. Milestones 2 & 5 — an implementation of the optimized Nitro protocol in Solidity. This will be in the form of a new version of the `[@statechannels/nitro-protocol](https://github.com/statechannels/statechannels/tree/master/packages/nitro-protocol)` NPM package.
3. Milestone 3 — an implementation of the optimized VFP in Typescript. This will exist in a new version of the `[@statechannels/wallet-core](https://github.com/statechannels/statechannels/tree/master/packages/wallet-core)` NPM package.
4. Milestone 4 — a formal verification of safety properties of the protocol using a combination of TLA+ (design) and Dafny (implementation).

What is not covered by the above milestones is the implementation of Nitro in Golang. The reference Typescript implementation will allow for Nitro to be ported to Golang and used in a Golang wallet. We may consider including an FIP to introduce Nitro state channels to Filecoin — see the optional Milestone 6 below.

Development Roadmap

Our anticipated project schedule can be found here

Comments on project schedule

- Our full team will not work on this project at all times. Our additional capacity may be applied to this project in the face of project delays.
- We are hitting the ground running, and have begun work on project already today, June 23!
- We have penciled in a block for a team meetup, pending changes to travel restrictions, extending the project schedule by an expected one week.
- Individual team members may be moved between projects for Milestones 3-5.

Milestone 1 — Optimized Nitro specification

Team: Mike Kerzhner, Andrew Stewart **Date**: Jun 23, 2021 → Jul 30, 2021 **Budget:** $10k USD

The optimal VFP has a TTP of 2 sequential messages, which we believe is the lower bound on “time to ready” for a multi hop channel — the recipient of a payment seems to need a signature from both the payer and the intermediary, necessitating two sequential one-way messages. This reduction in latency comes along with a reduction in the complexity and CPU overhead of the protocol, which we expect to be relevant to an active payment broker. It also significantly reduces the complexity and gas cost to dispute a virtual channel.

We will specify a version of Nitro protocol that enables streamlining the VFP by:

- removing guarantor channel `G`
- embedding application channel `X` into joint channel `J`
- enabling concurrent ledger update in `L` to include guarantee, before getting support in `J`

This requires making Nitro’s outcome format more flexible, to support heterogeneous types of outcome items. It also requires removing forced round-robin turn-taking semantics from the core protocol.

Existing work

- A detailed motivation for this milestone is in this document.
- A prototype “joint channel rules” implementation exists here which “embeds” an application channel.

Deliverables

We will specify the optimized Nitro protocol, including

- The state format, including the new exit format

- The adjudication process, including “dynamic turn taking” semantics
- The following outcome transformations
  - `claimGuarantees(ledgerOutcome, targetOutcome, indicesToClaim)`
    - A modification of the current behaviour of `claim` simplifies the construction of a virtual channel with multiple intermediaries, allowing for the safe construction with (apparently) constant latency. This claim must be confirmed with a more rigorous analysis.
  - `refundGuarantee(ledgerOutcome, indicesToRefund)`
    - Currently, it is not safe to direct fund to a channel that is not fully set up. This forces certain messages in the current VFP to be sequential. By adding a “refund” to a guarantee, we can remove the sequential requirement. This idea is explored in this issue.
  - `transfer(outcome, indicesToTransfer)`
    - We do not foresee that the current implementation of `transfer` needs to materially change

We will host this specification on https://docs.statechannels.org/.

We will specify a simplified off-chain VFP that achieves a TTP of 3. In addition, we will specify the optimized VFP protocol loosely specified here, including `validTransition` rules for a special case joint channel implementation required to reduce TTP to the conjectured minimum of 2.

Milestone 2: Updated Nitro Protocol’s “exit format”

Team: George Knee, Colin Kennedy **Date**: Jun 23, 2021 → Aug 20, 2021 **Budget**: $12k USD

During the EthGlobal hackathon, George explored the idea of a standardized “L2 exit format”, whose motivation can be found in the project readme. This format facilitates many optimizations of Nitro Protocol, in particular of the Virtual Funding Protocol, by allowing for more heterogeneous outcome types. One low-hanging fruit, as outlined here, is that the “guarantor” channels described in the Nitro paper are not necessary. Instead of separate channels, guarantees may be implemented as a piece of metadata on the outcome of the funding ledger channels, reducing the number of on-chain interactions required to challenge a virtual channel.

We will update the nitro-protocol package to use an outcome format that:

- Allows for heterogeneous outcome item types. This facilitates the removal of guarantor channels from the VFP
- Facilitates the addition of metadata to enable further Nitro optimizations.

We will use the outcome format defined in the “L2 exit format” repository as a starting point, adjusting it according to our more recent research & development.

Existing work

- A candidate exit format exists at https://github.com/statechannels/exit-format, with prototype implementations of asset transformations of a partially optimized VFP.

Deliverables

A detailed outline of this milestone can be found in this document.

1. Finalize an appropriate outcome format for optimized Nitro.
2. Update `[nitro-protocol](https://github.com/statechannels/statechannels/tree/master/packages/nitro-protocol)` & `[wallet-core](https://github.com/statechannels/statechannels/tree/master/packages/wallet-core)` use the new outcome format.
3. (Time dependent) The gas benchmarks for exiting a virtually funded channel (committed to the source code of that package) will reflect a reduced number of blockchain transactions (there will be no `challenge` `G`) and gas expenditure will decrease. A

stakeholder can verify this.
4. (Time dependent) Update `server-wallet` to use the new exit format.

Milestone 3: Optimized Virtual Funding Protocol implementation

Team: Mike Kerzhner, Alex Gap **Budget**: $8k USD **Date**: Aug 09, 2021 → Sep 10, 2021

We will implement the optimized off-chain Virtual Funding Protocol outlined here. At the end of this milestone, the optimized VFP will be available for use in Nitro wallet implementations.

We may implement two versions of the VFP:

1. With a simple "joint channel" implementation, we can implement a simple VFP with a TTP of 3 sequential messages.
2. With a more sophisticated "joint channel" implementation, we can reduce the TTP to 2 sequential messages.

The first sub-milestone is incremental towards the second.

Existing work

- Some prototype code is available for re-use in this file for the current VFP.

Deliverables

- Implement `virtual-funder` and `virtual-defunder` protocols in `wallet-core`.
  - These protocols specify which states states must be signed, and at what point is it safe to do so, similar to the "direct funder" protocol implemented here for directly funded channels.
  - The implementation will be accompanied by a test suite.
- Port the ledger funding protocol to `wallet-core`
  - Update its implementation to account for the ledger channel's outcome containing guarantees.
- Release a new version of `wallet-core`.

Milestone 4: Formal verification of optimized Nitro

Team* Andrew Stewart, George Knee **Date**: Aug 09, 2021 → Sep 24, 2021 **Budget:** $12k USD

***Note:** Franck Cassez or Joanne Fuller from the Trustworthy Smart Contracts team within ConsenSys Software R&D will assist us with some initial Dafny work.**

Adding refunds to the VFP widens the "surface area" for subtle vulnerabilities. We think that TLA+ is an appropriate tool for checking this area of the protocol. We can use our previous efforts with TLA+ to bootstrap this modeling.

TLA+ works best, however, with a simplified version of the protocol. One can verify that the full protocol emulates the simplified protocol by writing an implementation in Dafny which proves certain properties about `claim` and `refundGuarantee`.

Existing work

- Our `tla-specs` repo contains a pattern modeling the security assumption that Eve can do anything except sign states using Alice's private key, and that Eve can front-run any transaction that Alice submits to a transaction pool.

Deliverables

The exact deliverables will be updated based on the outcome of Milestone 1.

1. TLA+:
   - Extend the "adjudication" modeling in `[@statechannels/tla-specs](https://github.com/statechannels/tla-specs)` to include timing, allowing us to verify interactions between `claim`,

`challenge` and `refundGuarantee`
    - Identify an appropriate simplification of the outcome transformations specified in optimized Nitro.
    - Verify no loss of value in the VFP.
2. Dafny:
    - Implement `claim` and `refundGuarantee` in Dafny, verifying that the full implementation emulates the simplification in part 1.

Milestone 5: Optimized Nitro on the EVM

Team George Knee, Colin Kennedy **Date**: Sep 06, 2021 → Oct 01, 2021 **Budget**: $8k USD

As we may be slightly modifying the Nitro architecture to include some gas optimizations, the exact deliverables and timeline may depend on the output of Milestones 1, 2 and 4.

Deliverables

- Implement `claim`, `refundGuarantee` and `transfer` in Solidity, incorporating these into Nitro asset holders.
- Release in a new version of `@statechannels/nitro-protocol`.

(Optional) Milestone 6: FIP to add optimized Nitro to Filecoin

There are viable solutions to bridge Filecoin to the EVM, so we are not convinced that Filecoin channels must be updated to facilitate payments for a retrieval market. For instance, one may transact using a wrapped FIL token (such as Ren) in a EVM Nitro state channel on some layer 2 for Ethereum, for gas efficiency.

Nevertheless, implementing virtual channels natively in Filecoin may benefit both the storage and retrieval markets, and we are happy to consider this approach. To properly sketch out the requirements for this milestone, we feel it is best to discuss this with Filecoin project leaders. We may prefer to leave this extension for a future grant round.

Total Budget Requested

Our team is employed by Consensys Mesh. We are requesting a budget of $50k USD from the Filecoin Foundation to offset Consensys Mesh's investment in this research and development.

Maintenance and Upgrade Plans

We have a long-term commitment to maintaining & improving the Nitro state channel protocol. We plan to follow this project with:

- A working, PoC demonstration of the optimized VFP operating between three Nitro wallet nodes. (This will actually occur somewhat in parallel with the proposed project.)
- Further gas optimizations of our EVM Nitro implementation.
- An implementation of the efficient multi-hop VFP sketched out here.
- A prototype implementation of a "high speed rails" payment channel — this construction requires no computation on the payer's critical path, and little computation on the receiver's critical path, which may benefit retrieval in CPU-starved environments.

Team

Our team, Magmo, joined Consensys's R&D department in 2019. We are now employed by Consensys Mesh in the Applied R&D department.

Team Members

- Alex Gap — software engineer
- Andrew Stewart — team lead
- Colin Kennedy — software engineer
- Connie Wong — assistant project manager (part-time)
- George Knee — researcher
- Mike Kerzhner — software engineer

Team Website

https://statechannels.org/

Relevant Experience

We collectively have more than 15 years of experience thinking about state channels! As mentioned above, we successfully shipped a virtual funding protocol implementation along with Web3Torrent.

Our team has a healthy mix of academic backgrounds (two PhDs in probability theory and quantum computing, one Masters in Computer Science) as well as software backgrounds.

Our team has previous experience using TLA+ to specify the adjudication aspect of Nitro protocol, and found it fruitful for finding subtle griefing attacks. This work lead to a simpler and more secure protocol.

We are also good pals with Tom Close, who invented Nitro!

Team code repositories

Our active monorepo can be found at https://github.com/statechannels/statechannels. This includes browser-wallet & server-wallet implementations. Due to our team's recent focus, these implementations currently have limited interoperability.

We have also implemented some Nitro utilities in Rust in https://github.com/statechannels/native-utils. Time permitting, we will consider using Rust for implementing the off-chain protocol, as it facilitates wallet implementations in many tech stacks, similar to https://github.com/filecoin-project/filecoin-ffi.

Some preliminary work to Milestone 1 (L2 exit format) can be found in https://github.com/statechannels/exit-format.

Additional Information

- Our colleagues at Pegasys R&D have implemented a PoC retrieval market, and are a very valuable asset for us to validate assumptions against.
- We have implemented a server-wallet that is intended to be used in a NodeJS process. In parallel with this project, we will be building out infrastructure to stress test the wallet implementation's capacity to operate as a virtual funding intermediary.